Impact of Token-Level Aggregation on Molecular Property Prediction

Leah Rosen
leahros@post.bgu.ac.il

Nadav B. Liberty
nadavlib@post.bgu.ac.il

Abstract

Obtaining expressive molecule-level representations is essential for reliable property prediction. Earlier approaches relied on manually engineered molecular descriptors, whereas recent advances employ transformer architectures to learn representations directly from chemical notations such as SMILES (Simplified Molecular Input Line Entry System). Among these models, `MoLFormer` uses mean pooling for molecular embedding aggregation, which may obscure structurally important motifs. In this work, we investigate an alternative aggregation strategy based on a dedicated `[CLS]` summary token (`MoLFormer-[CLS]`) to produce a more informative sequence embedding. We conduct an extensive hyperparameter search and fine-tune the model on multiple MoleculeNet classification and regression benchmarks. Under resource-constrained training settings, `MoLFormer-[CLS]` surpasses the mean pooling baseline on key classification tasks (BBBP, BACE) and achieves competitive performance on the Lipophilicity (LIPO) regression task. Although results were more task-dependent, our findings suggest that `[CLS]`-based aggregation can yield more discriminative molecular representations, particularly for classification settings. Code availability: `https://github.com/leahrosen21/token-level-molecular-embedding-aggregation.git`

1 Introduction

Accurately predicting molecular properties from chemical structures is vital for accelerating drug discovery and materials science. While traditional handcrafted descriptors were once the standard, transformer-based [1] language models like `MoLFormer` [2] now learn these representations directly from SMILES strings. By treating chemical structures as sequences, these models capture complex structural information through self-attention.

Despite the success of this paradigm, the optimal strategy for condensing token-level embeddings into a single molecule-level representation remains poorly understood, especially as the ideal aggregation mechanism may depend heavily on the specific nature of the chemical task. `MoLFormer` uses mean pooling, which averages all token embeddings equally. However, this assumes every atom or bond contributes uniformly to a molecule's property, an assumption that may contradict the chemical reality where specific functional groups often dictate behavior and activity.

In contrast, standard large natural language models like `BERT` [3] and `RoBERTa` [4] utilize a dedicated `[CLS]` token as a learned global summary. This raises the question of whether uniform averaging is sufficient, or if a learnable aggregation mechanism can better capture chemical structure and nuance.

In this work, we systematically compare mean pooling against `[CLS]`-based aggregation. We evaluate our approach, `MoLFormer-[CLS]`, against the original `MoLFormer` across multiple MoleculeNet [5] benchmarks. Our results examine whether traditional natural language processing (NLP) architectural choices effectively translate to the chemical domain and if molecular property prediction requires task-specific aggregation strategies.

BGU IEM Deep Learning Course 2026 - Final Project.

2 Background

This study is based on the `MoLFormer` model, a transformer-based encoder-only architecture designed for molecular property prediction. `MoLFormer-XL`, the most robust variant, which we refer to here as `Molformer`, was pre-trained on a large corpus of 1.1 billion unlabeled molecules from the PubChem [6] and ZINC [7] datasets. Unlike traditional graph-based models, `MoLFormer` learns contextual relationships between atoms and substructures using SMILES strings, enabling more accurate and scalable predictions of molecular properties. The model demonstrated that pretraining on large molecular datasets and fine-tuning on downstream tasks can achieve competitive results on benchmarks such as MoleculeNet. The model can be adapted to multiple tasks, including classification and regression, achieving strong performance across datasets. To evaluate our approach, we use several MoleculeNet benchmark datasets covering various molecular properties. These include both classification and regression tasks:

Table 1: MoleculeNet datasets used in this study

Dataset	**Task Type**	**Samples**	**Description**
BBBP	Classification	2,039	Predicts blood-brain barrier permeability.
BACE	Classification	1,513	Predicts binding of inhibitors to β-secretase 1.
ClinTox	Classification	1,478	Prediction of clinical trial toxicity of drugs.
ESOL	Regression	1,128	Predicts water solubility of molecules.
FreeSolv	Regression	642	Predicts hydration free energy of small molecules.
Lipophilicity (LIPO)	Regression	4,200	Predicts fat solubility of molecules.

3 Contributions and Methodology

To address the comparison against mean pooling, we developed and explored an alternative aggregation framework based on the `[CLS]` token. As stated above, while mean pooling averages all token embeddings across a sequence, our approach leverages the specialized representation at the beginning of the input string. During fine-tuning, the embedding of this token learns to aggregate the most task-relevant contextual and structural information from the entire molecular sequence. We utilized the pre-trained `MoLFormer` model sourced from the Hugging Face platform. In the `MoLFormer` tokenizer, the summary token is represented as `<bos>`. For a SMILES string such as `OCC(O)C(O)CO`, the tokenized sequence ($\mathcal{S}$) is represented as:

$$\mathcal{S} = [\ \underbrace{\texttt{<bos>}}_{\text{Summary Token}}, \text{O, C, C, (, O,), C, (, O,), C, O,}\ \underbrace{\texttt{<eos>}}_{\text{End}}, \underbrace{\texttt{<pad>}, \ldots, \texttt{<pad>}}_{\text{Padding}}]$$

The proposed framework follows a four-stage pipeline. The SMILES sequence is tokenized and passed to the encoder. The sequence is then processed by the pre-trained `MoLFormer` encoder to obtain contextualized hidden-state embeddings. The final hidden-state embedding corresponding to the first token is extracted and used as a molecular representation. This representation is subsequently fed into a task-specific classification head to produce the final prediction. To ensure the integrity of our comparison, we utilized the exact classification head architecture as defined in the official `MoLFormer` implementation. This allowed us to better isolate the effect of the `[CLS]` embedding strategy versus the traditional mean pooling approach. Listing 1 illustrates the core architectural modification. Complete implementation details are available in the project repository.

In addition to using the standard MoleculeNet benchmarks, we implemented a custom `PyTorch` dataset class to ensure consistent pre-processing across all experiments. Each SMILES string was first validated and canonicalized using RDKit [8]. Invalid molecular notations were discarded to prevent downstream numerical instability during prediction. Canonical SMILES sequences were generated to enforce a unique structural encoding of each molecule. All sequences were tokenized using the model's tokenizer and padded or truncated to a fixed maximum length of 202 tokens, matching the model's input constraints.

```python
class CustomModelForSequenceClassification(nn.Module):
    def __init__(self, model_name, num_labels):
        super().__init__()
        self.encoder = AutoModel.from_pretrained(model_name,
            trust_remote_code=True)
        self.classifier = MolformerClassificationHead(
            self.encoder.config, num_labels)

    def forward(self, input_ids, attention_mask):
        outputs = self.encoder(input_ids, attention_mask)
        cls_embeddings = outputs.last_hidden_state[:, 0, :]
        logits = self.classifier(cls_embeddings)
        return {"loss": loss, "logits": logits}
```

Listing 1: `PyTorch` implementation of the `[CLS]`-based sequence classification architecture.

Fine-tuning was performed using the Hugging Face `transformers` library, specifically leveraging the `Trainer` and `TrainingArguments` classes. This setup allowed for a highly reproducible and modular training pipeline [9]. We initially fine-tuned the `MoLFormer-[CLS]` model for each task using hyperparameters similar to those used in the original `MoLFormer` study. To further improve performance, we conducted a comprehensive hyperparameter search for all tasks. All experimental metrics, including loss and evaluation scores, were logged using the Weights & Biases (W&B) [10] platform for real-time tracking and analysis.

4 Results

In this section, we present and analyze the results obtained from fine-tuning `MoLFormer-[CLS]` on the downstream classification and regression tasks, as well as the outcomes of our hyperparameter exploration studies. We begin by examining the initial fine-tuning experiments conducted under preliminary training settings and hyperparameters, which slightly differ from the `MoLFormer` benchmark configuration. Particularly, the experiments were limited to 10 epochs as an initial validation of the fine-tuning procedure. Therefore, the results should be interpreted as a rough performance indication rather than a fully controlled benchmark comparison. The W&B-logged project for these experiments cannot be made public due to platform restrictions.

Table 2: Initial fine-tuning test set results for the classification tasks (F_1 and ROC-AUC %).

Dataset	**Loss**	**F_1 Score**	**ROC-AUC** (`MoLFormer-[CLS]`)	**ROC-AUC** (`MoLFormer`)
BBBP	0.372	90.2	92.01	**93.70**
BACE	0.433	73.9	87.29	**88.20**
ClinTox	0.216	22.2	91.36	**94.80**

Table 3: Initial fine-tuning results on the test set for the regression tasks.

Dataset	**Loss**	**MAE**	**RMSE** (`MoLFormer-[CLS]`)	**RMSE** (`MoLFormer`)
FreeSolv	7.451	2.153	2.730	**0.2308**
ESOL	0.937	0.751	0.968	**0.2787**
LIPO	0.407	0.484	0.638	**0.5298**

4.1 Initial Fine-tuning

The initial fine-tuning results demonstrate that `MoLFormer-[CLS]` achieves competitive performance across both classification and regression benchmarks. For the classification tasks (Table 2), strong ROC-AUC scores are observed for BBBP and ClinTox, both exceeding 91%, indicating effective molecular representation learning even under preliminary training settings. Although ClinTox achieves strong test performance, validation ROC-AUC is lower (76.6%), potentially reflecting

dataset characteristics such as class imbalance or structural heterogeneity. Overall performance remains competitive, however, ensuring consistent metrics may require further analysis of dataset properties and representation behavior. Performance on BACE is comparatively lower, possibly indicating task-specific challenges that should be further explored.

For the regression tasks (Table 3), our proposed approach demonstrates reasonable predictive accuracy on ESOL and LIPO, with relatively low MAE and RMSE values. FreeSolv appears more challenging, as indicated by higher error values, which may be due to its smaller dataset size (642 samples). As in the classification setting, the benchmark `MoLFormer` configuration outperforms the `[CLS]`-based variant, though the performance gap varies across tasks and may stem from the difference in fine-tuning settings. Generally, these results indicate that the proposed fine-tuning framework is stable and effective, providing solid baseline performance prior to hyperparameter optimization.

4.2 Hyperparameter Search

To assess sensitivity to optimization settings and improve performance consistency, we conducted a random hyperparameter search. Validation performance was evaluated across 10 runs varying learning rate, batch size, warmup ratio, and weight decay. Each configuration was trained for one epoch to allow efficient comparison, and logged via W&B. The search space is summarized in Table 4. In contrast to our single-epoch search configuration, the `MoLFormer` fine-tuning setup employed a batch size of 128, an initial learning rate of 3×10^{-5}, 8 data-loading workers, and a maximum of 500 training epochs.

Table 4: Hyperparameter search space.

Hyperparameter	**Search Range**
Batch Size (Train)	{8, 16, 32, 64}
Learning Rate	Log-uniform: $[10^{-5}, 3 \times 10^{-5}]$
Weight Decay	{0.0, 0.1, 0.2, 0.3, 0.4, 0.5}
Warmup Ratio	{0.0, 0.1, 0.2, 0.3}
Fixed Parameters	Data-loading Workers: 1, Max Epochs: 1

Table 5: Hyperparameter search results for the BBBP classification task using `MoLFormer-[CLS]`.

Run	**Learning Rate**	**Batch Size**	**Warmup Ratio**	**Weight Decay**	**Loss**	**F_1 score (%)**	**ROC-AUC (%)**
1	2.19×10^{-5}	16	0.1	0.4	0.391	89.0	90.3
2	2.91×10^{-5}	8	0.3	0.0	0.379	**89.1**	**92.7**
3	2.75×10^{-5}	16	0.3	0.3	**0.373**	88.5	91.3
4	1.38×10^{-5}	64	0.2	0.4	0.519	82.8	84.0
5	1.03×10^{-5}	8	0.3	0.3	0.389	88.1	90.7
6	2.95×10^{-5}	16	0.1	0.4	0.387	88.1	91.1
7	1.53×10^{-5}	16	0.0	0.5	0.403	88.1	89.3
8	1.41×10^{-5}	16	0.2	0.1	0.411	88.4	88.9
9	1.45×10^{-5}	64	0.3	0.5	0.515	83.5	84.7
10	1.67×10^{-5}	64	0.2	0.1	0.486	83.9	86.2

4.3 Optimal Task-Specific Hyperparameter Selection

For brevity, detailed search logs are provided only for BBBP (Table 5) and LIPO (Table 6) as representative examples. Table 7 summarizes the selected configurations for all tasks.

For classification, BBBP displayed a consistent advantage for smaller batch sizes (8), which improved F_1 and ROC-AUC despite batch size of 16 achieving slightly lower validation loss (0.373 vs. 0.379). Higher learning rates combined with moderate weight decay (0.3) and warmup (0.3) were most effective. BACE exhibited similar behavior. Although the highest ROC-AUC (85.7%) for this task was achieved without warmup, we adopted a conservative 0.1 warmup ratio for stability. ClinTox, likely due to severe class imbalance (91 positive vs. 1094 negative samples), was sensitive to hyperparameters, and only specific configurations produced non-zero F_1 scores. The selected setup achieved a ROC-AUC of 72.46% while ensuring stable training.

Table 6: Hyperparameter search results for the LIPO regression task using `MoLFormer-[CLS]`.

Run	Learning Rate	Batch Size	Warmup Ratio	Weight Decay	Loss	MAE	RMSE
1	2.48×10^{-5}	8	0.0	0.4	**0.449**	**0.523**	**0.670**
2	2.79×10^{-5}	32	0.1	0.4	0.568	0.604	0.754
3	1.44×10^{-5}	8	0.2	0.3	0.537	0.575	0.733
4	1.18×10^{-5}	16	0.1	0.0	0.677	0.663	0.823
5	1.21×10^{-5}	16	0.1	0.2	0.670	0.660	0.819
6	2.42×10^{-5}	32	0.1	0.1	0.598	0.624	0.774
7	1.86×10^{-5}	32	0.3	0.5	0.650	0.650	0.806
8	1.67×10^{-5}	64	0.2	0.2	0.804	0.718	0.897
9	1.71×10^{-5}	16	0.0	0.4	0.596	0.623	0.772
10	2.13×10^{-5}	16	0.3	0.1	0.540	0.588	0.735

For regression tasks, performance was mainly sensitive to batch size and learning rate. A batch size of 8 consistently produced the lowest MAE and RMSE, while larger batches degraded performance. Optimal learning rates were in the range of 2×10^{-5}-3×10^{-5}, and moderate weight decay (0.1-0.4) provided the best trade-off between stability and generalization. Although more extreme regularization settings occasionally yielded competitive results, we favored moderately regularized configurations to ensure stable optimization.

Table 7: Selected hyperparameter settings for classification and regression tasks.

Task Type	Dataset	Learning Rate	Batch Size	Warmup Ratio	Weight Decay
Classification	BBBP	2.91×10^{-5}	8	0.3	0.3
	BACE	2.62×10^{-5}	8	0.1	0.5
	ClinTox	1.82×10^{-5}	16	0.2	0.4
Regression	FreeSolv	2.96×10^{-5}	8	0.2	0.1
	ESOL	2.85×10^{-5}	8	0.3	0.4
	LIPO	2.48×10^{-5}	8	0.1	0.4

4.4 Results of Improved Fine-tuning

We executed a finalized fine-tuning phase using the optimized hyperparameters, testing each configuration across varied durations of up to 70 epochs. Recall that the original `MoLFormer` was likely subjected to more extensive training, and despite our computational limitations, the `MoLFormer-[CLS]` model demonstrates superiority over the baseline in certain tasks. This suggests that the aggregation strategy and hyperparameter refinement compensate for the limited training.

Table 8: Final fine-tuning test set results for the classification tasks (F_1 and ROC-AUC %).

Dataset	Loss	F_1 Score	ROC-AUC (`MoLFormer-[CLS]`)	ROC-AUC (`MoLFormer`)
BBBP	0.666	92.6	**94.61**	93.70
BACE	0.591	76.7	**90.11**	88.20
ClinTox	0.492	38.1	89.11	**94.80**

Table 8 summarizes the optimal performance achieved for classification tasks. These results represent the models selected based on a balance between test set performance and validation stability. After extending the training (up to 20 epochs for BBBP, 10 for BACE), the models achieved ROC-AUC scores of 94.61% and 90.11% respectively. Both of these results exceed the standard `MoLFormer` benchmarks, demonstrating the efficacy of the `[CLS]` aggregation approach on these datasets. For the ClinTox dataset, we observed a trade-off between absolute performance and stability. While a run at 10 epochs yielded a higher ROC-AUC of 91.95% (with an F_1 of 31.6% and loss of 0.356), it demonstrated a lower validation ROC-AUC of approximately 82.3%. We ultimately favored the 20-epoch run, which provided a more stable validation ROC-AUC of 86.9% and a superior F_1 score of 38.1%, despite the slightly lower test ROC-AUC of 89.11%.

Table 9: Final fine-tuning results on the test set for the regression tasks.

Dataset	**Loss**	**MAE**	**RMSE** (`MoLFormer-[CLS]`)	**RMSE** (`MoLFormer`)
FreeSolv	1.32	0.792	1.1495	**0.2308**
ESOL	0.318	0.427	0.5637	**0.2787**
LIPO	0.266	0.374	**0.5159**	0.5298

In Table 9, we present the best-performing models on the regression datasets. Our experiments suggest that additional training epochs are required for full convergence, with ESOL and LIPO likely benefiting from at least 30 epochs and FreeSolv from 50 or more. LIPO, despite being the largest dataset, achieved competitive results after 20 epochs (RMSE 0.5159, loss 0.266). ESOL reached an RMSE of 0.5637 and loss of 0.3177 after 20 epochs, improving over the initial fine-tuning stage, though not surpassing the `MoLFormer` benchmark. FreeSolv, a fundamentally different task, achieved an RMSE of 1.1495 and loss of 1.32 after 50 epochs. Notably, its RMSE appeared to converge a value of 1, and extending training to 70 epochs did not yield meaningful improvement. Therefore, we report the 50-epoch model. Overall, further analysis is needed to better understand the behavior and effectiveness of our approach on regression tasks.

The best performing models, even those that do not surpass the baseline, are provided in the project repository under the filename `{dataset_name}_best_model.zip`.

5 Conclusions and future directions

In this study, we evaluated `[CLS]`-based aggregation as an alternative to mean pooling in the `MoLFormer` architecture, using a dedicated `[CLS]` token to preserve structural information more effectively for downstream tasks. Our results show that its effectiveness is task-dependent. It proved particularly beneficial for certain classification tasks, while regression tasks required longer optimization or potentially alternative aggregation strategies. Among the regression benchmarks, LIPO achieved competitive performance, indicating that the approach can extend beyond classification under suitable conditions. Recall that these results were obtained with limited computational resources and shorter training schedules than the original `MoLFormer` study.

Although exposed to a mild ceiling effect, the performance on classification tasks, especially BBBP and BACE, is unlikely to be incidental. These tasks may rely more on specific localized structural motifs rather than averaged global properties. The `[CLS]` token assigns greater weight to such critical substructures, whereas mean pooling may attenuate their contribution. This distinction may explain the advantage observed in classification settings.

In contrast, for regression tasks, our approach proved less consistently effective. These tasks may inherently require more distributed molecular representations rather than a single global summary. While solubility-related datasets such as ESOL and LIPO appear to align reasonably well with our approach, our findings suggest that regression tasks generally demand more extensive training than classification. Moreover, a single `[CLS]` summary token may not always be sufficient to capture the continuous property variations required. Further investigation is necessary to determine whether the observed limitations arise from a representational bottleneck introduced by the `[CLS]` token, the intrinsic complexity of the tasks, or the relatively limited training schedule.

Several future directions could further strengthen the `MoLFormer-[CLS]` framework. Improving interpretability through attention map analysis may clarify which substructures the `[CLS]` token emphasizes. Applying dimensionality reduction methods such as t-SNE [11] or UMAP [12] could also reveal whether `[CLS]`-based aggregation yields more separable property-specific representations than mean pooling. An additional emergent direction is parameter-efficient fine-tuning (PEFT), particularly LoRA [13], which we began exploring using the `peft` integration in the `transformers` API. Finally, broader hyperparameter searches and longer training schedules would help determine whether the advantages of the `[CLS]` strategy persist under more extensive optimization.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[2] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties. *Nature Machine Intelligence*, 4(12):1256–1264, 2022.

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[4] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[5] Zhenqin Wu, Bharath Ramsundar, Evan N Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S Pappu, Karl Leswing, and Vijay Pande. Moleculenet: a benchmark for molecular machine learning. *Chemical science*, 9(2):513–530, 2018.

[6] Sunghwan Kim, Jie Chen, Tiejun Cheng, Asta Gindulyte, Jia He, Siqian He, Qingliang Li, Benjamin A Shoemaker, Paul A Thiessen, Bo Yu, et al. Pubchem 2019 update: improved access to chemical data. *Nucleic acids research*, 47(D1):D1102–D1109, 2019.

[7] John J Irwin and Brian K Shoichet. Zinc- a free database of commercially available compounds for virtual screening. *Journal of chemical information and modeling*, 45(1):177–182, 2005.

[8] G Landrum. Rdkit: open-source cheminformatics http://www. rdkit. org. *Google Scholar There is no corresponding record for this reference*, 3(8), 2016.

[9] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, pages 38–45, 2020.

[10] Lukas Biewald. Experiment tracking with weights and biases, 2020. Software available from wandb.com.

[11] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

[12] Leland McInnes, John Healy, and James Melville. Umap: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.

[13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.